



Sistemas Interactivos y Ubicuos

Universidad Carlos III de Madrid

Entregable P2: Prototipar

Curso 2025/2026

Grupo: 81

Autores:

David Mateu Serrano (100522337)

Manuel Manchado Barquero (100522228)

Fran Jurado Basté (100522213)

Índice

1. Introducción	2
1.1. Descripción del sistema	2
1.2. Objetivos del prototipo	2
2. Especificaciones técnicas	3
2.1. Arquitectura del sistema	3
2.1.1. Diagrama de componentes	3
2.1.2. Diagrama de comunicación	4
2.2. Tecnologías utilizadas	7
3. Implementación de los prototipos	7
3.1. Interacciones implementadas	7
3.1.1. Navegar y seleccionar	7
3.1.2. Control de funciones principales	7
3.1.3. Salida o confirmación	8
3.1.4. Funcionalidades adicionales	8
3.2. Demostración en vídeo	9
4. Prototipado experiencial	9
4.1. Sesiones realizadas	9
4.2. Observaciones y ajustes	9
4.3. Vídeos de las sesiones y análisis de hallazgos	10
5. Iteraciones y mejoras	10
5.1. Problemas técnicos encontrados	10
5.2. Cambios respecto al diseño original (P1)	11
5.3. Nuevas ideas generadas	11
6. Reflexión sobre el uso de IA	11
7. Reflexión final	11

1. Introducción

El presente documento recoge el proceso de implementación correspondiente a la fase de prototipado (P2) del proyecto de Sistemas Interactivos y Ubicuos, enmarcado en la metodología del *Design Sprint Kit* de Google.

El sistema desarrollado, está orientado a tenistas amateurs que entrenan sin entrenador y necesitan acceder a información relevante del partido —marcador, feedback técnico y avisos de falta— sin interrumpir el flujo de juego ni depender de interacciones táctiles. Esta necesidad fue identificada en la fase P1 a partir del *Experience Map* y los *Pain Points* del usuario, y validada físicamente mediante sesiones de *Bodystorming*.

1.1. Descripción del sistema

Como esta representado el sistema:

Tenis Inteligente es un sistema interactivo ubicuo orientado al tenista amateur que entrena sin entrenador. Su objetivo es proporcionar información relevante del partido —marcador, estadísticas tácticas y feedback técnico— de forma no intrusiva y sin requerir que el jugador suelte la raqueta ni interactúe táctilmente con ningún dispositivo.

El sistema opera sobre dos dispositivos físicos: un **una pantalla en el banquillo** que actúa como de *dashboard* principal y un **teléfono móvil** insertado en la raqueta del jugador (en el modo entrenamiento se separará el sensor para grabar al usuario), que actúa como controlador de sensores.

El sistema ofrece dos modos de funcionamiento principales, seleccionables por voz:

- **Modo Partido:** el jugador controla el marcador mediante gestos de giro de muñeca. El sistema registra automáticamente estadísticas tácticas por punto (winners, errores, potencia de golpe, tipo de golpe), muestra el marcador de una forma que resulta muy familiar a los jugadores de tenis (puntos, juegos y sets, incluyendo tie-break), y avisa de cambios de pista.
- **Modo Entrenamiento:** el móvil activa la cámara y envía landmarks corporales via Socket.IO a la computadora, que los procesa con MediaPipe para analizar la biomecánica del jugador. Incluye tres sub-modos seleccionables por voz: **fondo** (desafío de 5 golpes desde el fondo con detección de la posición del codo), **Saque** (desafío de 5 golpes con detección de extensión de codo y altura de impacto) y **línea** (detección de falta de pie mediante un modelo entrenado de Teachable Machine).

Toda la comunicación entre dispositivos se realiza en tiempo real mediante **Socket.IO**, con el servidor Node.js actuando como *broker* central que recibe eventos del móvil y los redistribuye al dashboard, y viceversa. Las estadísticas del partido se persisten en el servidor en un fichero, permitiendo recuperar el estado tras una reconexión.

1.2. Objetivos del prototipo

El prototipo desarrollado en esta fase persigue los siguientes objetivos:

- Validar técnicamente la viabilidad de las interacciones definidas en P1, comprobando que los gestos de giro de muñeca y los comandos de voz funcionan de forma fiable en un entorno real de uso.
- Implementar una arquitectura distribuida funcional que permita la comunicación en tiempo real entre el móvil del jugador y el dashboard del portátil, sin dependencia de servicios externos.
- Demostrar que es posible controlar un marcador de tenis completo —con lógica de puntos, juegos, sets y tie-break— sin ningún contacto táctil, únicamente mediante gestos corporales y voz.
- Explorar la viabilidad de usar la cámara del móvil como sensor biomecánico, procesando landmarks de MediaPipe en tiempo real para dar feedback técnico al jugador sobre su golpe o su saque.
- Detectar automáticamente la falta de pie en el saque mediante visión artificial, eliminando la necesidad de un juez de línea en entrenamientos individuales.
- Establecer una base funcional sobre la que realizar las pruebas con usuarios reales previstas en la fase P3.

2. Especificaciones técnicas

2.1. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor centralizada en la que el servidor Node.js actúa como *broker* de eventos entre los dos clientes web: el **dashboard** (portátil) y el **controlador** (móvil). Ningún cliente se comunica directamente con el otro; toda la mensajería pasa por el servidor mediante Socket.IO sobre WebSocket.

2.1.1. Diagrama de componentes

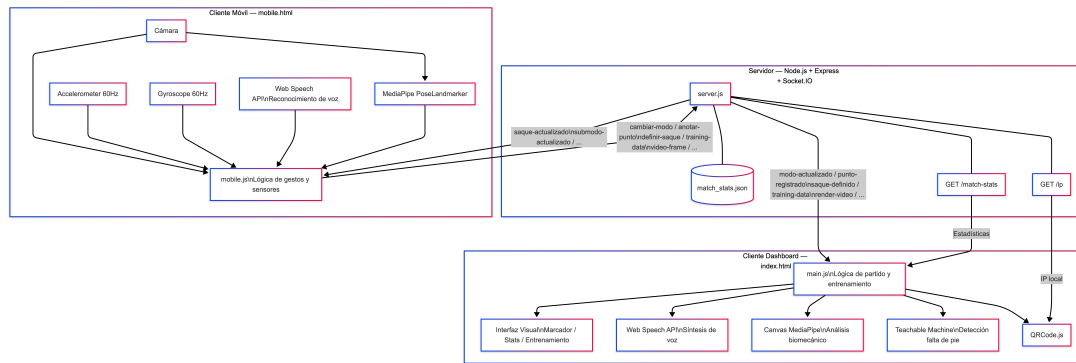


Figura 1: Diagrama de componentes del sistema.

El sistema se compone de tres capas diferenciadas:

- **Servidor (Node.js + Express + Socket.IO):** gestiona las conexiones de ambos dispositivos, actúa como intermediario de todos los eventos, mantiene el estado global del modo activo (`modoActual`, `subModoActual`) y persiste las estadísticas del partido en `match_stats.json`. Expone además dos endpoints REST: `/ip` para la generación del QR y `/match-stats` para recuperar estadísticas tras una reconexión.
- **Cliente Dashboard (portátil):** interfaz visual principal que muestra el marcador en vivo, el panel de estadísticas tácticas y la vista de entrenamiento con el canvas de MediaPipe. Recibe eventos del servidor y actualiza la UI en consecuencia. Gestiona también la síntesis de voz (*Web Speech API*) para los avisos sonoros al jugador y gestiona todo lo relacionado con procesamiento de Teachable Machine y Media pipe tras recibir la información captada desde el controlador.
- **Cliente Controlador (móvil):** captura los datos de los sensores inerciales del dispositivo (acelerómetro y giroscopio a 60 Hz), detecta gestos y golpes, procesa los landmarks de pose con MediaPipe, y emite los eventos correspondientes al servidor. Incorpora también el reconocimiento de voz para la selección de modo y los comandos de control.

2.1.2. Diagrama de comunicación

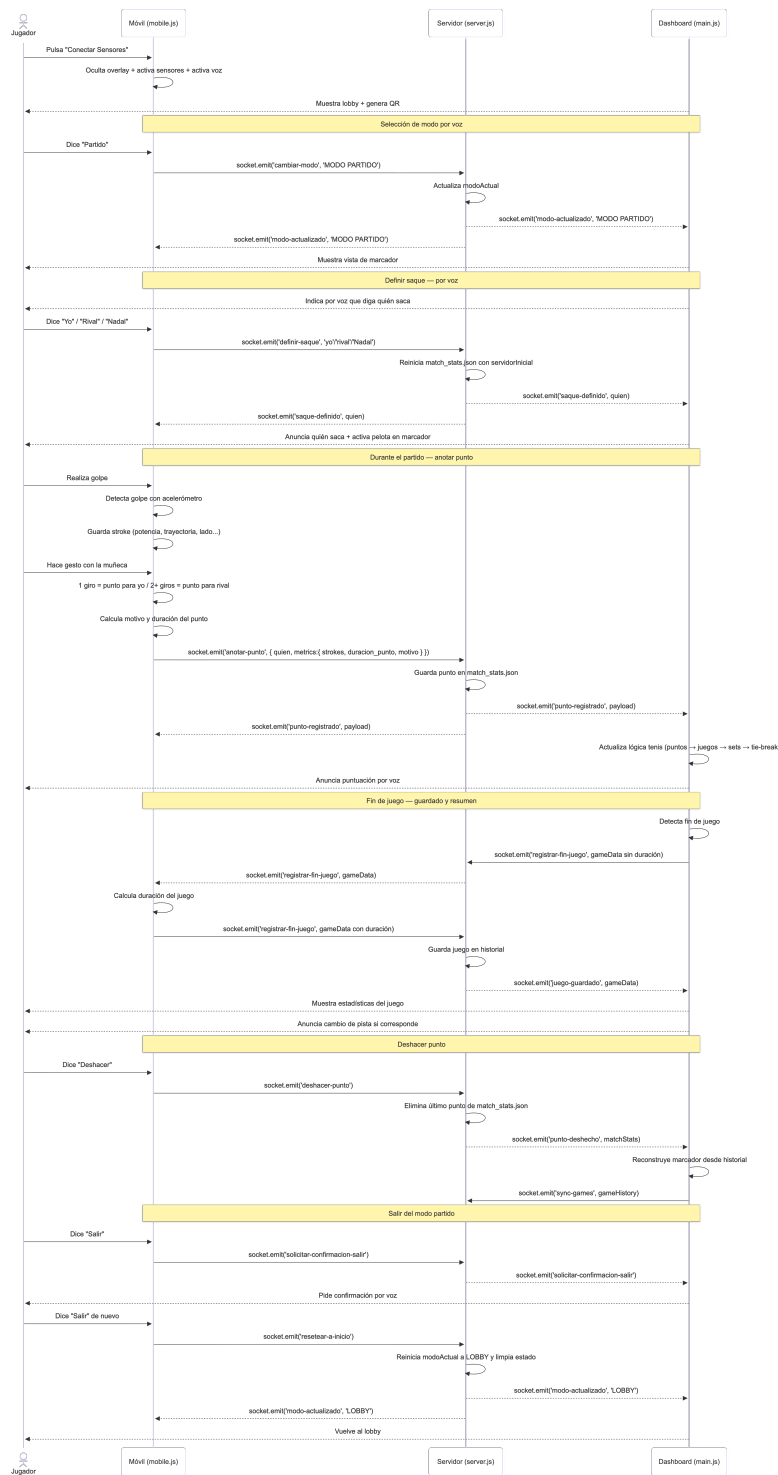


Figura 2: Flujo de eventos Socket.IO — Modo Partido.

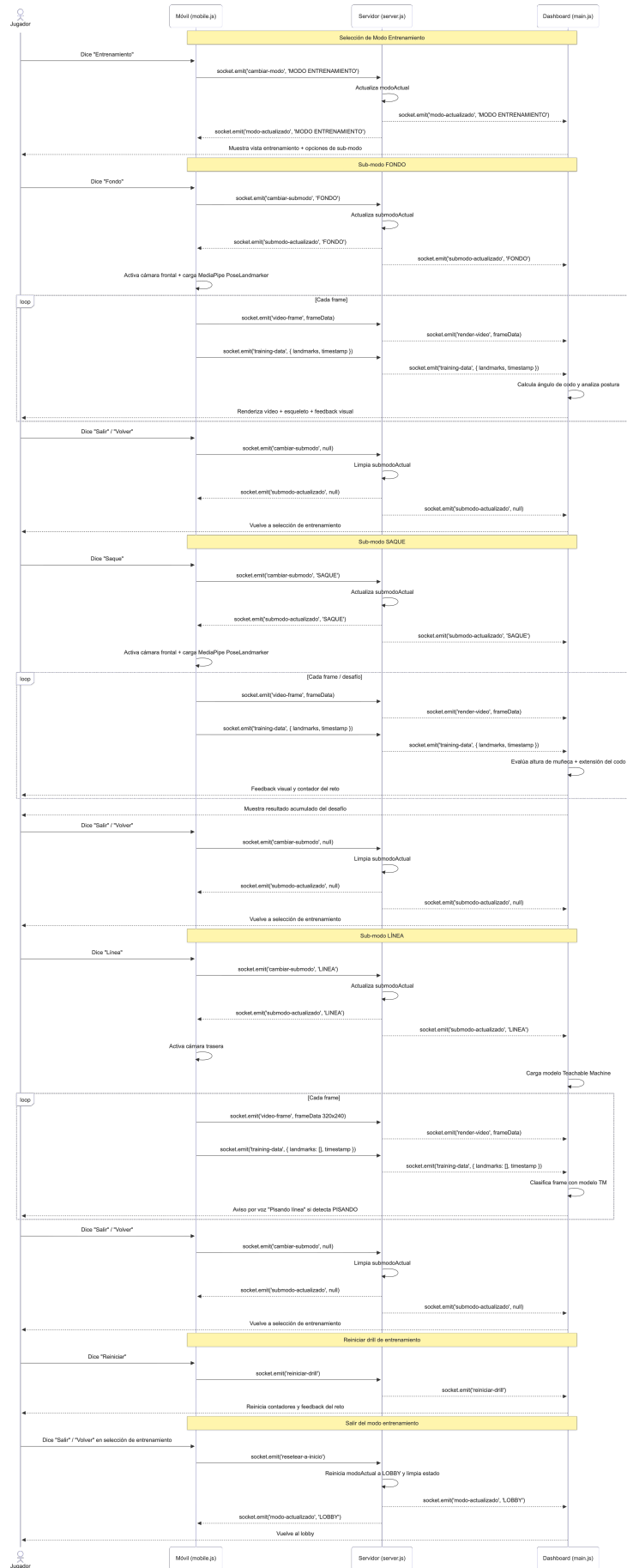


Figura 3: Flujo de eventos Socket.IO — Modo Entrenamiento.

Los principales eventos que circulan por el canal Socket.IO son los siguientes:

Evento	Dirección	Descripción
cambiar-modo	Móvil → Srv → Dashboard	Selección de Modo Partido o Entrenamiento mediante voz.
definir-saque	Móvil → Srv → Dashboard	Establece quién saca al inicio del partido; reinicia <code>match_stats.json</code> .
anotar-punto	Móvil → Srv → Dashboard	Registra un punto con métricas (potencia, tipo de golpe, trayectoria, duración).
punto-registrado	Servidor → Dashboard	Notifica al dashboard el punto anotado para actualizar la lógica de puntos, juegos y sets.
registrar-fin-juego	Dashboard → Servidor	Notifica al servidor el fin de un juego con sus métricas acumuladas.
deshacer-punto	Móvil → Srv → Dashboard	Cancela el último punto. El dashboard reconstruye el marcador iterativamente.
punto-deshecho	Servidor → Dashboard	Historial actualizado tras eliminar el último punto.
sync-games	Dashboard → Servidor	Sincroniza el historial de juegos recalculado tras una reconstrucción.
notificar-saque	Dashboard → Srv → Móvil	Informa al móvil del cambio de servidor tras cada juego.
saque-actualizado	Servidor → Móvil	Confirma al móvil quién tiene el saque tras un cambio de juego.
training-data	Móvil → Srv → Dashboard	Envía landmarks de pose (MediaPipe) para análisis biomecánico en tiempo real.
video-frame	Móvil → Srv → Dashboard	Transmite frames de vídeo (320×240) para Teachable Machine en modo Línea.
cambiar-submodo	Móvil → Srv → Dashboard	Activa el sub-modo de entrenamiento (Fondo, Saque, Línea) o lo desactiva (<code>null</code>).
submodo-actualizado	Servidor → Móvil + Dashboard	Notifica a ambos dispositivos el sub-modo activo.
alternar-audio	Móvil → Srv → Dashboard	Silencia o activa la síntesis de voz del dashboard.
solicitar-confirmacion-salir	Móvil → Srv → Dashboard	Solicita confirmación por voz antes de salir del modo activo.
resetear-a-inicio	Móvil → Srv → Dashboard	Reinicia el sistema al lobby limpiando el estado y las estadísticas en memoria.

Cuadro 1: Eventos Socket.IO del sistema y su dirección de propagación.

2.2. Tecnologías utilizadas

Tecnología	Uso en el sistema
Node.js + Express	Servidor HTTP que sirve los ficheros estáticos, expone los endpoints REST (/ip, /match-stats) y gestiona el estado global del partido.
Socket.IO	Canal de comunicación bidireccional en tiempo real entre el móvil y el dashboard. Permite emitir y escuchar eventos con latencia mínima sobre WebSocket.
MediaPipe Tasks Vision	Detección de landmarks corporales (33 puntos de pose) en el navegador del móvil. Se usa para analizar el ángulo del codo, la altura de la muñeca y la postura durante los modos Fondo y Saque.
Teachable Machine (TensorFlow.js)	Modelo de clasificación de imágenes entrenado para detectar si el jugador pisa la línea durante el saque (Modo Línea). Se carga desde la URL remota del modelo.
Web Speech API (reconocimiento)	Reconocimiento de voz en el móvil para la selección de modo ("Partido", "Entrenamiento"), comandos de control ("Salir", "Cancelar") y selección de sub-modo.
Web Speech API (síntesis)	Síntesis de voz en el dashboard para los avisos sonoros al jugador: puntuación, cambio de pista, feedback técnico y confirmaciones.
Sensor APIs (Accelerometer / Gyroscope)	Lectura de acelerómetro lineal y giroscopio a 60 Hz en el móvil para detectar golpes (por magnitud de aceleración) y giros de muñeca (por velocidad angular en el eje Y). Se aplica un filtro de media móvil de 4 muestras para reducir ruido.
QRCode.js	Generación del código QR en el dashboard con la URL local del controlador móvil, obtenida dinámicamente desde el endpoint /ip.

Cuadro 2: Tecnologías utilizadas y su rol en el sistema.

3. Implementación de los prototipos

3.1. Interacciones implementadas

A continuación se describen y justifican cada una de las interacciones implementadas en el prototipo, organizadas según las funcionalidades mínimas requeridas y las adicionales desarrolladas por el equipo.

3.1.1. Navegar y seleccionar

La navegación entre modos se realiza íntegramente mediante comandos de voz, sin necesidad de contacto táctil. Desde el móvil, el reconocimiento de voz (*Web Speech API*) escucha de forma continua y detecta las palabras clave "Partido" y "Entrenamiento" para seleccionar el modo activo. Una vez en el Modo Entrenamiento, el jugador puede seleccionar el sub-modo deseado pronunciando "Fondo", "Saque" o "Línea".

Esta decisión de diseño responde directamente a la HMW de P1 sobre interacción manos libres: durante el juego el jugador tiene la raqueta y las pelotas en la mano, por lo que cualquier interacción táctil resulta inviable.

3.1.2. Control de funciones principales

Modo Partido. El control principal del sistema durante el Modo Partido se articula sobre dos mecanismos gestuales capturados por los sensores inerciales del móvil:

- **Detección de golpe (acelerómetro):** el sistema calcula la magnitud del vector de aceleración lineal en los tres ejes. Cuando supera el umbral de 18 m/s^2 , se registra un golpe. Para evitar falsos positivos

provocados por el ruido eléctrico del sensor, se aplica un filtro de media móvil de 4 muestras antes de evaluar el umbral. Se impone además un tiempo mínimo de 850 ms entre golpes consecutivos.

- **Giro de muñeca (giroscopio):** el sistema monitoriza la velocidad angular en el eje Y del giroscopio. Cuando supera los 10 rad/s con una aceleración simultánea baja (indicativo de que el brazo no se está moviendo con fuerza, sino girando suavemente), se interpreta como un gesto de puntuación. Un giro suma un punto al jugador; dos giros consecutivos en una ventana de 1000 ms suman un punto al rival.

El dashboard mantiene en todo momento la lógica completa del tenis: gestiona la progresión de puntos (0, 15, 30, 40, ventaja), juegos, sets y tie-break. Tras cada punto, la puntuación se anuncia automáticamente por voz. Al finalizar cada juego, el panel lateral de estadísticas se actualiza mostrando los datos tácticos acumulados: winners y errores desglosados por tipo de golpe (derecha/revés), tasa de error, porcentaje de primer saque, aces, dobles faltas, potencia media y pico máximo del partido, y tiempos medios por punto y por juego.

Modo Entrenamiento — Sub-modo Fondo. En el sub-modo Fondo, el móvil activa la cámara frontal y envía los landmarks corporales al dashboard mediante MediaPipe *PoseLandmarker*. El dashboard calcula en tiempo real el ángulo del codo a partir de los vectores hombro-codo y codo-muñeca, y lo muestra numéricamente sobre el canvas junto al esqueleto del jugador. El sistema propone un desafío de 5 golpes y evalúa en cada uno si la extensión del brazo en el momento del impacto es correcta.

Modo Entrenamiento — Sub-modo Saque. El sub-modo Saque sigue la misma arquitectura de captura que el Fondo, pero el análisis se centra en dos métricas específicas del gesto de saque: la altura de la muñeca dominante en el punto de impacto y el ángulo de extensión del codo. El sistema detecta el ciclo de preparación (codo doblado) seguido del momento de impacto (codo extendido y muñeca elevada), y evalúa si el jugador alcanza la altura y la extensión recomendadas. Al completar el desafío de 5 saques, el dashboard muestra el resultado final con el número de repeticiones correctas sobre el total.

3.1.3. Salida o confirmación

La salida del modo partido requiere una doble confirmación por voz para evitar salidas accidentales durante el juego. Al pronunciar “*Salir*” por primera vez, el sistema emite el evento `solicitar-confirmacion-salir` al dashboard, que solicita confirmación en voz alta. Si el jugador repite “*Salir*” en los siguientes segundos, se ejecuta el evento `resetear-a-inicio` y el sistema vuelve al lobby. En caso contrario, la petición se cancela automáticamente. Para el resto de modos, no existe esta doble confirmación.

Adicionalmente, el comando “*Deshacer*” permite deshacer el último punto anotado en cualquier momento, emitiendo el evento `deshacer-punto`. El servidor elimina el último registro de `match_stats.json` y el dashboard reconstruye el marcador iterativamente desde el historial completo de puntos.

3.1.4. Funcionalidades adicionales

Detección de falta de pie (Modo Línea). El sub-modo Línea activa la cámara trasera del móvil y transmite frames de vídeo al dashboard mediante el evento `video-frame`. A diferencia de los sub-modos Fondo y Saque, que envían landmarks de pose, en este caso se transmiten frames de vídeo comprimidos a 320×240 píxeles para reducir el ancho de banda consumido. El dashboard los clasifica en tiempo real con un modelo de *Teachable Machine* (TensorFlow.js) entrenado específicamente para distinguir si el pie del jugador pisa o no la línea de saque. El modelo se entrena con imágenes capturadas desde la perspectiva de la cámara trasera del móvil situado en la zona de saque, con dos clases: *pie dentro* y *falta de pie*. Cuando la predicción supera el umbral de confianza establecido, el dashboard emite un aviso sonoro inmediato mediante síntesis de voz, eliminando la necesidad de un juez de línea en entrenamientos individuales. Para evitar una avalancha de avisos repetidos, se aplica un semáforo (`isProcessingTM`) que bloquea nuevas predicciones hasta que la actual se resuelve.

Estadísticas tácticas en tiempo real. Cada punto anotado se almacena en el servidor con un conjunto de métricas capturadas por el acelerómetro: potencia del golpe (magnitud del pico de aceleración en m/s^2), trayectoria estimada (plana, liftada o cortada según el perfil temporal del golpe) y zona de golpeo (derecha o revés). El panel lateral del

dashboard agrega estas métricas y muestra en tiempo real el total de winners, errores no forzados, tasa de error, porcentaje de primer saque, aces, dobles faltas, potencia media y pico máximo. Los valores se actualizan por punto y por juego, mostrando deltas respecto al último descanso para facilitar la lectura táctica.

Silenciar y activar la síntesis de voz. El dashboard utiliza la *Web Speech API* de síntesis para verbalizar automáticamente la puntuación tras cada punto, avisar de cambios de pista, anunciar el inicio del tie-break y proporcionar feedback técnico tras cada golpe en el Modo Entrenamiento. Dado que en determinados contextos el audio puede resultar molesto o innecesario, el jugador puede silenciar o reactivar este canal en cualquier momento pronunciando “*Silenciar*” o “*Activar*” desde el móvil. El evento *alternar-audio* se propaga a través del servidor para sincronizar el estado entre ambos dispositivos, de forma que el dashboard confirma el cambio verbalizando “*Sistema silenciado*” o “*Sonido activado*” antes de aplicar la nueva configuración.

Easter egg — Modo Nadal. Como elemento no planificado en P1, el sistema incorpora un modo oculto activable mediante un comando de voz específico: si el jugador se identifica como Rafael Nadal, el sistema reconoce su nombre y reproduce un vídeo sorpresa en pantalla completa sobre el dashboard. Este modo se implementó como un guiño al contexto del proyecto y no interfiere con ninguna funcionalidad del sistema, ya que se ejecuta como un overlay temporal sobre la interfaz principal.

3.2. Demostración en vídeo

El vídeo de demostración del sistema está disponible en el siguiente enlace:

- <https://youtu.be/h0h32Pp02dI>

4. Prototipado experiencial

4.1. Sesiones realizadas

El equipo llevó a cabo sesiones de prototipado experiencial a lo largo del desarrollo, en las que cada miembro adoptó de forma rotatoria los roles de usuario, sistema y observador, siguiendo la misma dinámica de *bodystorming* aplicada en P1. El objetivo en esta fase fue distinto al de P1: mientras que en la fase de ideación el *bodystorming* sirvió para descartar ideas inviables, en P2 las sesiones se orientaron a detectar problemas concretos de la implementación y calibrar los parámetros de los sensores.

Se realizaron principalmente dos tipos de sesiones:

- **Sesiones de calibración gestual:** pruebas con el sistema funcionando en las que se ajustaron los umbrales del acelerómetro y el giroscopio. El equipo simuló situaciones reales de juego —golpes de derecha, revés y giros de muñeca— para comprobar si el sistema distinguía correctamente entre un golpe real y un movimiento accidental.
- **Sesiones de flujo completo:** pruebas del sistema de extremo a extremo en las que un miembro simulaba un partido completo mientras los otros observaban la respuesta del dashboard y tomaban nota de los fallos o comportamientos inesperados.

4.2. Observaciones y ajustes

Las sesiones de prueba iterativas revelaron varios problemas que llevaron a ajustes directos sobre la implementación:

- **Falsos positivos en la detección de golpe:** el umbral inicial de aceleración era demasiado bajo, lo que provocaba que movimientos bruscos del brazo sin impacto (como preparar el saque o recoger una pelota) fueran interpretados como golpes. Se elevó el umbral a 18 m/s^2 y se introdujo el filtro de media móvil de 4 muestras, lo que redujo significativamente los falsos positivos.

- **Confusión entre golpe y giro de muñeca:** en las primeras versiones, el giroscopio detectaba giros de muñeca durante golpes potentes, sumando puntos de forma involuntaria. Se resolvió añadiendo la condición de baja aceleración simultánea (magnitud $< 20 \text{ m/s}^2$) como requisito para que un giro sea interpretado como gesto de puntuación.
- **Salidas accidentales del modo activo:** durante las pruebas, el comando “Salir” se activaba al hablar con otros jugadores si la conversación contenía esa palabra. Se implementó el mecanismo de doble confirmación descrito en la sección anterior para mitigar este problema.
- **Latencia en el Modo Entrenamiento:** el envío de frames de vídeo completos al servidor generaba una latencia perceptible en el canvas del dashboard. Se redujo la resolución de los frames enviados a 320×240 píxeles y se introdujo un semáforo (`isProcessingTM`) para evitar la acumulación de predicciones del modelo de Teachable Machine.
- **Reconocimiento de voz en entornos ruidosos:** el reconocimiento de voz mostraba una tasa de error elevada al probarlo en espacios con ruido ambiental. Se acotó el vocabulario activo al mínimo necesario en cada estado del sistema, de forma que el reconocedor solo escucha las palabras relevantes para el modo activo en cada momento.

4.3. Vídeos de las sesiones y análisis de hallazgos

Los vídeos de las sesiones de prototipado experiencial están disponibles en:

- https://youtu.be/-9noUc_hSh0
- <https://youtu.be/C4cMqhckMfg>

Los hallazgos más relevantes de estas sesiones pueden resumirse en dos conclusiones. En primer lugar, la interacción gestual basada en sensores inerciales es viable pero extremadamente sensible a los umbrales de calibración: pequeñas variaciones en la forma de sujetar el móvil o en la intensidad del golpe pueden alterar el comportamiento del sistema. En segundo lugar, la voz resulta un canal de entrada más robusto que los gestos para acciones de navegación, pero requiere un vocabulario reducido y un contexto de uso silencioso o semisilencioso para funcionar con fiabilidad.

5. Iteraciones y mejoras

5.1. Problemas técnicos encontrados

A lo largo del desarrollo se identificaron diversos problemas técnicos que requirieron decisiones de diseño o cambios en la implementación original:

- **Acceso a los sensores inerciales en iOS:** la *Sensor API* (Accelerometer, Gyroscope) no está disponible en Safari, el navegador por defecto de iPhone. Esto limitó el uso del móvil como controlador a dispositivos Android con Chrome, condicionando el entorno de pruebas del equipo.
- **Permisos de dispositivo en Chrome para Android:** el acceso al giroscopio y al acelerómetro requiere que el servidor sirva el contenido bajo HTTPS o desde `localhost`. Durante las pruebas en red local, el móvil accedía al servidor por IP, lo que bloqueaba los permisos. Se resolvió añadiendo la cabecera de permisos adecuada en Express y documentando este requisito en el `README.md`.
- **Sincronización del marcador tras deshacer punto:** la primera aproximación consistía en almacenar el estado del marcador directamente en el servidor y restar un punto al deshacer. Sin embargo, esto generaba inconsistencias en la lógica de sets y tie-breaks. Se optó por un enfoque de *event sourcing* ligero: el servidor solo almacena el historial de puntos en `match_stats.json`, y el dashboard reconstruye el marcador completo de forma iterativa cada vez que se deshace un punto.
- **Implementación de IA** intentamos utilizar un modelo de IA llamado Qwen2.5-0.5B para que tomase los datos de los jugadores durante el partido y aportase feedback sobre cómo mejorar tu técnica etc. El

problema es que no conseguimos que el modelo genere respuestas realmente valiosas para el usuario.

- **Gestos y navegación** En nuestro diseño el gesto de golpear se parecía mucho al gesto de sumar punto por lo que los sensores no identificaban la diferencia por lo que tuvimos que decidirnos por un giro de muñeca para sumar punto.

5.2. Cambios respecto al diseño original (P1)

La implementación final difiere en algunos aspectos respecto a las ideas definidas en la fase de convergencia de P1:

- **Semáforo de tiempo:** esta funcionalidad, seleccionada en el Dot Voting de P1, no fue implementada en el prototipo actual. Durante el desarrollo se priorizó la estabilidad del marcador y el Modo Entrenamiento, dejando el semáforo como mejora pendiente para P3.
- **Ampliación del Modo Entrenamiento:** el diseño original de P1 no contemplaba sub-modos de entrenamiento diferenciados. La implementación incorporó tres sub-modos (Fondo, Saque y Línea) como resultado de las sesiones de prototipado experiencial, que pusieron de manifiesto que un único modo genérico no era suficiente para dar feedback útil en contextos tan distintos.
- **Persistencia de estadísticas:** P1 no preveía ningún mecanismo de almacenamiento. Durante el desarrollo se identificó la necesidad de persistir el historial de puntos para soportar el deshacer y la recuperación tras reconexión, lo que llevó a implementar la escritura en `match_stats.json`.

5.3. Nuevas ideas generadas

Las sesiones de prototipado y el proceso de desarrollo han generado ideas que podrían incorporarse en futuras iteraciones del sistema:

- Implementar el semáforo de tiempo pendiente de P1, mostrando un contador visual en el dashboard que avise al jugador cuando se acerca el límite de tiempo entre saques reglamentario.
- Exportar el informe de estadísticas del partido en formato PDF al finalizar, permitiendo al jugador revisar su evolución entre sesiones.
- Implementar sistema de logros.

6. Reflexión sobre el uso de IA

El equipo ha utilizado herramientas de IA generativa, principalmente **Claude** (Anthropic) como apoyo durante el desarrollo del proyecto. Claude se empleó para la estructuración y redacción de la memoria en $\text{\LaTeX}$ y para resolver dudas sobre compatibilidad de las Web APIs entre navegadores. Gemini se usó como asistente de codificación para autocompletar fragmentos repetitivos de JavaScript y gestión de eventos Socket.IO.

En ambos casos, las sugerencias fueron revisadas y adaptadas por el equipo antes de incorporarse al proyecto. Las decisiones de diseño, la arquitectura del sistema y la calibración de los sensores fueron realizadas íntegramente por los miembros del grupo.

7. Reflexión final

El desarrollo de este prototipo ha supuesto un paso significativo en la materialización de las ideas definidas en P1. El proceso de implementación ha confirmado que las interacciones diseñadas son técnicamente viables, aunque ha puesto de manifiesto que la distancia entre un concepto de diseño y un sistema funcional es considerable: aspectos que parecían triviales en papel, como la distinción entre un golpe y un giro de muñeca o el reconocimiento de voz en entornos ruidosos, han requerido iteraciones significativas para funcionar con fiabilidad.

El principal aprendizaje del equipo ha sido la importancia de probar el sistema en condiciones reales desde las primeras fases del desarrollo. Las sesiones de prototipado experiencial fueron determinantes para detectar problemas que no habrían emergido en un entorno de pruebas controlado, y permitieron tomar decisiones de diseño fundamentadas en evidencia física en lugar de suposiciones.

De cara a P3, el sistema presenta una base funcional sólida sobre la que realizar las pruebas con usuarios reales. Los aspectos que consideramos más relevantes a validar son la curva de aprendizaje de los gestos para usuarios no familiarizados con el sistema, la fiabilidad del reconocimiento de voz en una pista de tenis real y la utilidad percibida de las estadísticas tácticas durante el juego. Los resultados de esta validación determinarán qué aspectos del sistema deben refinarse antes de considerar el prototipo como una solución madura.